AI ASSISTED CODING

NAME: P Sree Varshini

BATCH: 40

ROLL NO : 2303A52336

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

• Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Instructions:

• Ask AI to locate repeated computations within the program.

• Encapsulate repeated logic into reusable functions.

• Verify that program output remains unchanged after refactoring.

• Add proper docstrings to all defined functions.

Sample Code:

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Expected Output:

A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

Lab_13.2.py > ...

```python
#TASK - 1
#restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
# Refactored Code:
def calculate_area(length, width):
    return length * width
def calculate_perimeter(length, width):
    return 2 * (length + width)
length = 8
width = 4
print("Area:", calculate_area(length, width))
print("Perimeter:", calculate_perimeter(length, width))
length = 12
width = 6
print("Area:", calculate_area(length, width))
print("Perimeter:", calculate_perimeter(length, width))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS Python Del

```
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC> & 'c:\Users\chand\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\chand\.v
Perimeter: 24
Area: 72
Perimeter: 36
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

ANALYSIS

- **The program calculates the area and perimeter of a rectangle.**
- **Initially, the same calculations are repeated in the code.**
- **Functions calculate_area() and calculate_perimeter() are created to avoid repetition.**
- **This makes the code reusable, cleaner, and easier to maintain.**

Task Description -2 (Refactoring – Modularizing Inline

Calculations)

• Task: Use AI to refactor a Python script by extracting repeated

inline calculations into reusable and well-documented functions.

Instructions:

• Identify repeated calculation patterns in the code.

• Convert them into reusable functions.

• Ensure readability and proper documentation.

Sample Code:

amount = 1000

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

amount = 2000

discount = amount * 0.10

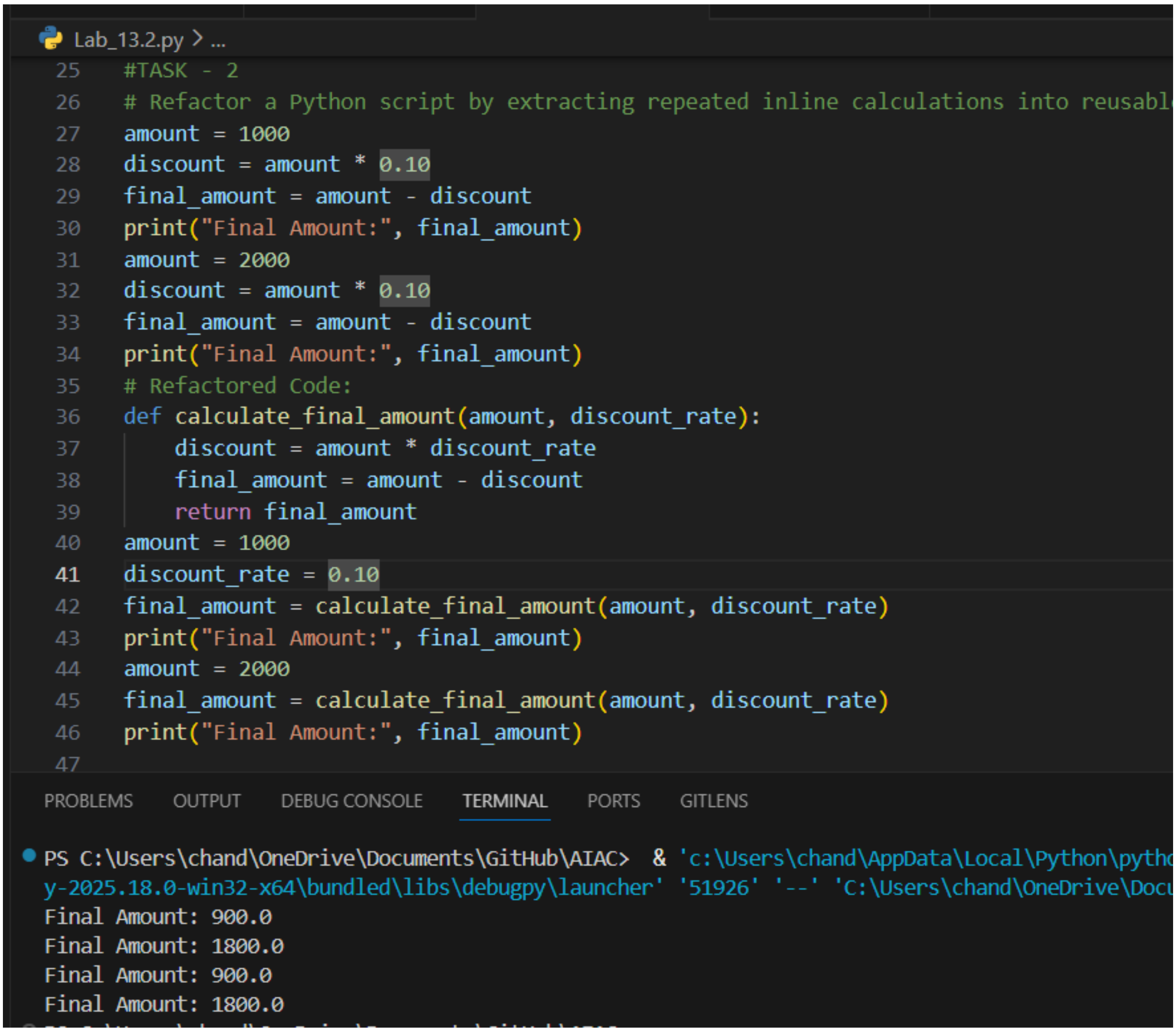
final_amount = amount - discount

print("Final Amount:", final_amount)

Expected Output:

A modular Python program that uses a reusable function to compute

final amounts for different inputs.



ANALYSIS

- **The program calculates the final amount after a discount.**
- **Initially, the same discount calculation is repeated in the code.**
- **A function calculate_final_amount() is created to avoid repetition.**
- **This makes the code cleaner, reusable, and easier to maintain.**

Task Description -3 (Refactoring – Improving Loop and

Conditional Performance)

• Task: Apply AI-based suggestions to enhance the performance of

Python code that relies on nested loops and repeated conditional

checks.

Instructions:

• Use AI to recommend more efficient data structures or logic

simplifications.

• Replace inefficient constructs while preserving program behavior.

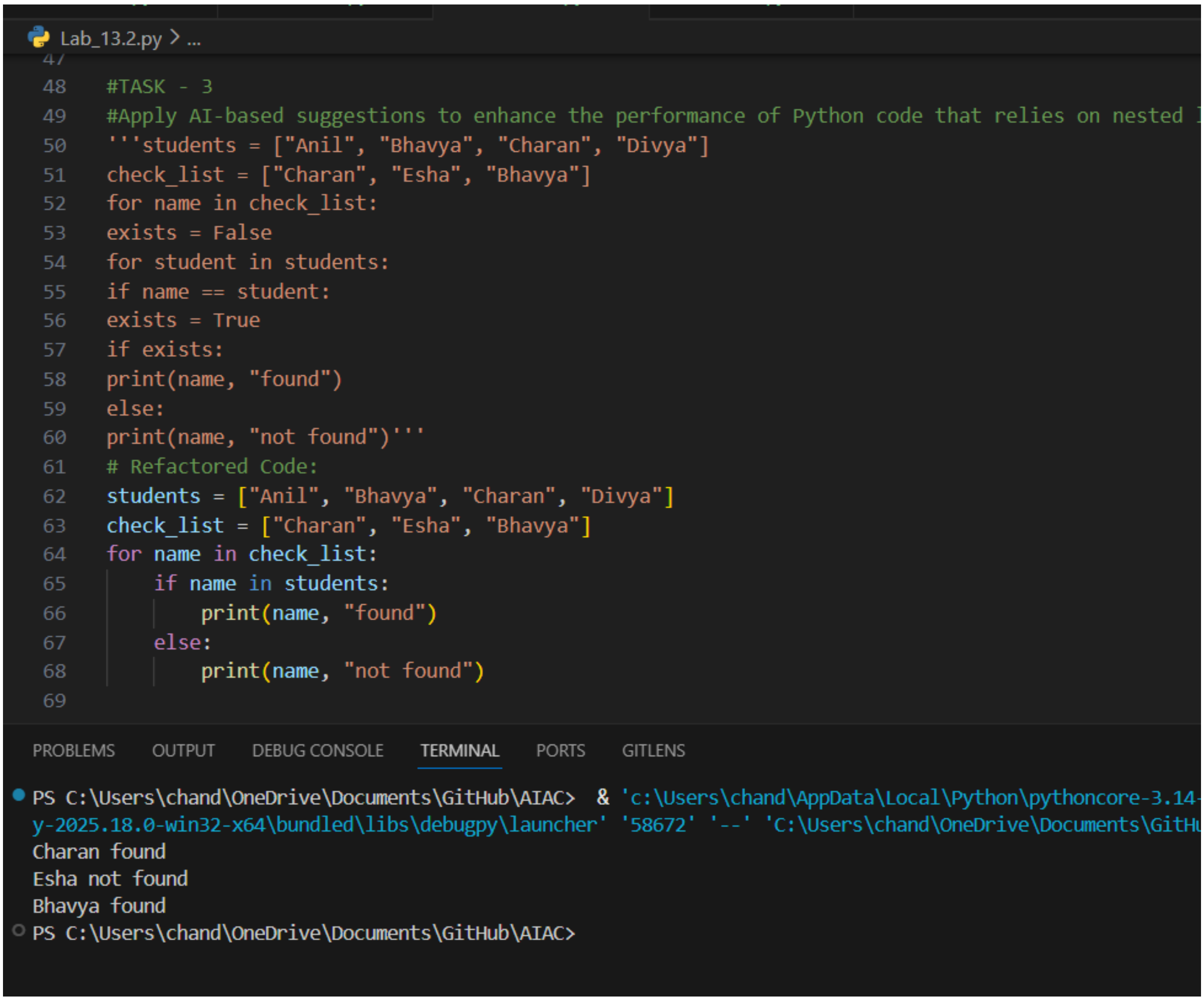
• Compare execution efficiency before and after refactoring.

Sample Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
for name in check_list:
exists = False
for student in students:
if name == student:
exists = True
if exists:
print(name, "found")
else:
print(name, "not found")
```

Expected Output:

An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.



ANALYSIS

- **The program checks whether names in check_list exist in the students list.**
- **Initially, it uses nested loops, which is slower.**
- **The refactored code uses if name in students to check directly.**
- **This improves performance and makes the code simpler.**

Task Description -4 (Refactoring – Replacing Hardcoded Values

with Constants)

• Task: Utilize AI assistance to improve maintainability by

replacing hardcoded numerical values with named constants.

Instructions:

• Detect hardcoded numeric values used directly in expressions.

• Define descriptive constants at the beginning of the program.

• Update calculations to use these constants without changing

results.

Sample Code:

print("Simple Interest:", (1000 * 2 * 5) / 100)

print("Simple Interest:", (2000 * 2 * 5) / 100)

Expected Output:

A revised Python program using constants such as RATE and TIME for

clearer and maintainable calculations..

Lab_13.2.py > ...

```python
#TASK - 4
#improve maintainability by replacing hardcoded numerical values with named constants.
print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100)
# Refactored Code:
PRINCIPAL_1 = 1000
PRINCIPAL_2 = 2000
RATE = 2
TIME = 5
def calculate_simple_interest(principal, rate, time):
    return (principal * rate * time) / 100
print("Simple Interest for Principal 1:", calculate_simple_interest(PRINCIPAL_1, RATE, TIME))
print("Simple Interest for Principal 2:", calculate_simple_interest(PRINCIPAL_2, RATE, TIME))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC>  & 'c:\Users\chand\AppData\Local\Python\pythoncore-3.14-64\py
y-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52482' '--' 'C:\Users\chand\OneDrive\Documents\GitHub\AIA
Simple Interest: 100.0
Simple Interest: 200.0
Simple Interest for Principal 1: 100.0
Simple Interest for Principal 2: 200.0
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC>
```

ANALYSIS

- The program calculates **Simple Interest**.
- Initially, it uses **hardcoded values** in the formula.
- The refactored code uses **named constants and a function** calculate_simple_interest().
- This makes the code **clearer, reusable, and easier to maintain**.

Task Description -5 (Re factoring – Enhancing Variable Naming

and Code Clarity)

• Task: Apply AI tools to improve code readability by renaming

unclear variables and adding meaningful comments..

Instructions:

• Replace non-descriptive variable names with meaningful

identifiers.

• Add inline comments where the logic may be unclear.

• Ensure program output remains the same.

Sample Code:

x = 50

y = 30

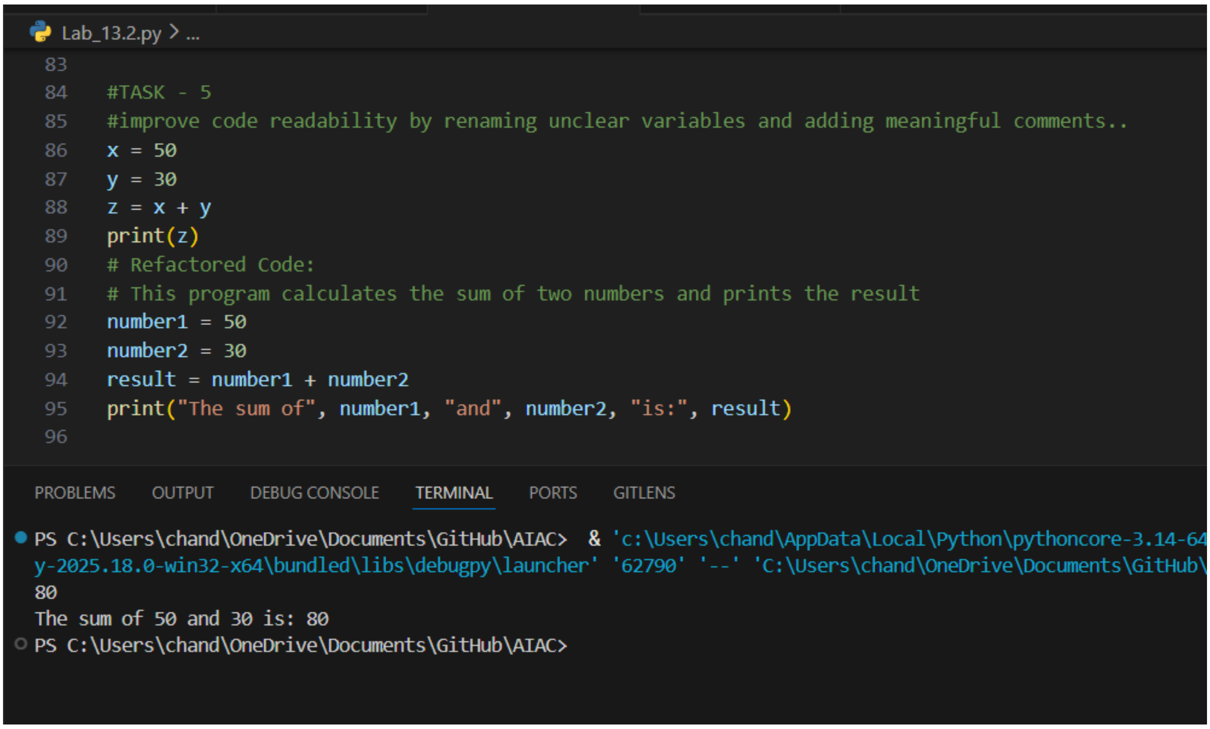
z = x + y

print(z)

Expected Output:

A Python program with clear variable names and explanatory comments

that improve readability without altering functionality.



ANALYSIS

- The program **adds two numbers and prints the result**.
- Initially, **unclear variable names** (x, y, z) are used.
- The refactored code uses **meaningful variable names** and comments.
- This **improves readability and understanding of the code**.